

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab_02_)

Implementation of Combinational Circuits using Verilog Gate & Dataflow Level Modeling

Student Name: ___Areeb Fatima___ Roll No: _____24I-6543_____ Section: _B_

Date performed: _____2nd September_____, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES, ISLAMABAD

Prepared by: Fasih Ahmad

Version: 1.0

Task 1:

Description:

In this first task, we were asked to write the half adder and full adder Verilog code by using the Gate level and Data Flow modeling. So, at Gate level we simply XOR for sum and AND for carry and full adder is made by using the two half adders and one OR gate. In Data Flow, we simply assign sum by using XOR of three inputs and assign carry to the AND of inputs and OR them. Similarly, we made the circuit diagram in Logisim.

Code:

HALF ADDER (Gate level):

```
module half_adder(A,B,sum,carry);  
  
input A,B;  
  
output sum,carry;  
  
xor(sum,A,B);  
  
and(carry,A,B);  
  
endmodule;
```

HALF ADDER (Data Flow):

```
module half_adder(A,B,sum,carry);  
  
input A,B;  
  
output sum,carry;  
  
assign sum= A^B;  
  
assign carry= A&B;  
  
endmodule;
```

Common Testbench (HALF ADDER):

```
module tb_ha();  
  
reg x,y;  
  
wire sum,carry;  
  
half_adder uut(x,y,sum,carry);  
  
initial  
  
begin  
  
x=0; y=0;  
  
#50;  
  
x=0; y=1;  
  
#50;  
  
x=1; y=0;  
  
#50;  
  
x=1; y=1;  
  
end  
  
endmodule;
```

FULL ADDER (Gate Level):

```
module half_adder(A,B,sum,carry);  
  
input A,B;  
  
output sum,carry;  
  
assign sum= A^B;  
  
assign carry= A&B;  
  
endmodule;  
  
module full_adder(a,b,CIN,SUM,CARRY);
```

```
input a,b,CIN;

output SUM,CARRY;

wire t1,t2,t3;

half_adder ha1(a,b,t1,t2);

half_adder ha2(t1,CIN,SUM,t3);

or o1(CARRY,t2,t3);

endmodule;
```

FULL ADDER (Data Flow):

```
module full_adder(a,b,CIN,SUM,CARRY);

input a,b,CIN;

output SUM,CARRY;

assign SUM= a^b^CIN;

assign CARRY= (a&b) | (b&CIN) | (CIN&a);

endmodule;
```

Common Testbench (FULL ADDER):

```
module tb_fa();

reg x,y,z;

wire sum,carry;

full_adder uut(x,y,z,sum,carry);

initial

begin

x=0; y=0; z=0;

#50;
```

```
x=0; y=0; z=1;

#50;

x=0; y=1; z=0;

#50;

x=0; y=1; z=1;

#50;

x=1; y=0; z=0;

#50;

x=1; y=0; z=1;

#50;

x=1; y=1; z=0;

#50;

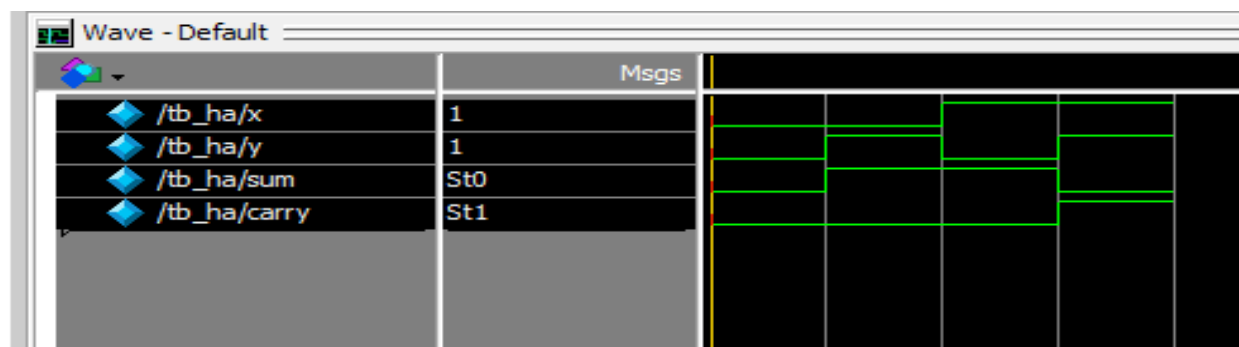
x=1; y=1; z=1;

end

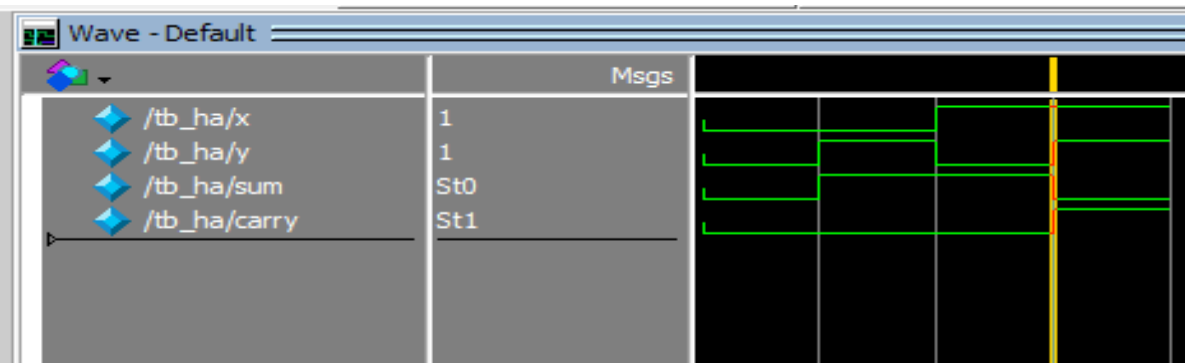
endmodule;
```

Output:

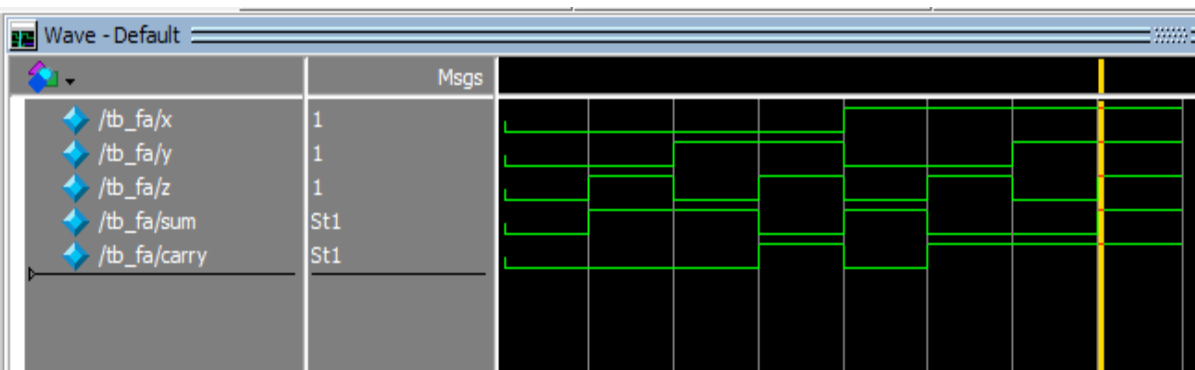
HALF ADDER (Gate level):



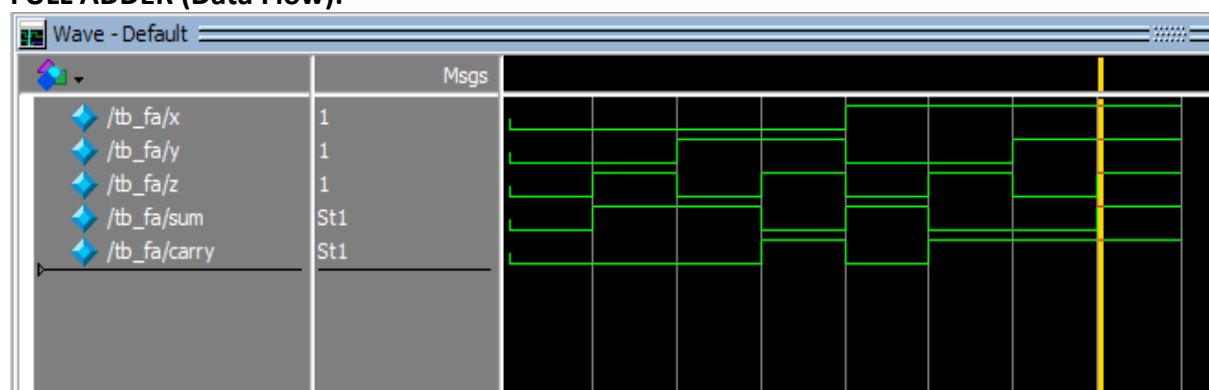
HALF ADDER (Data Flow):



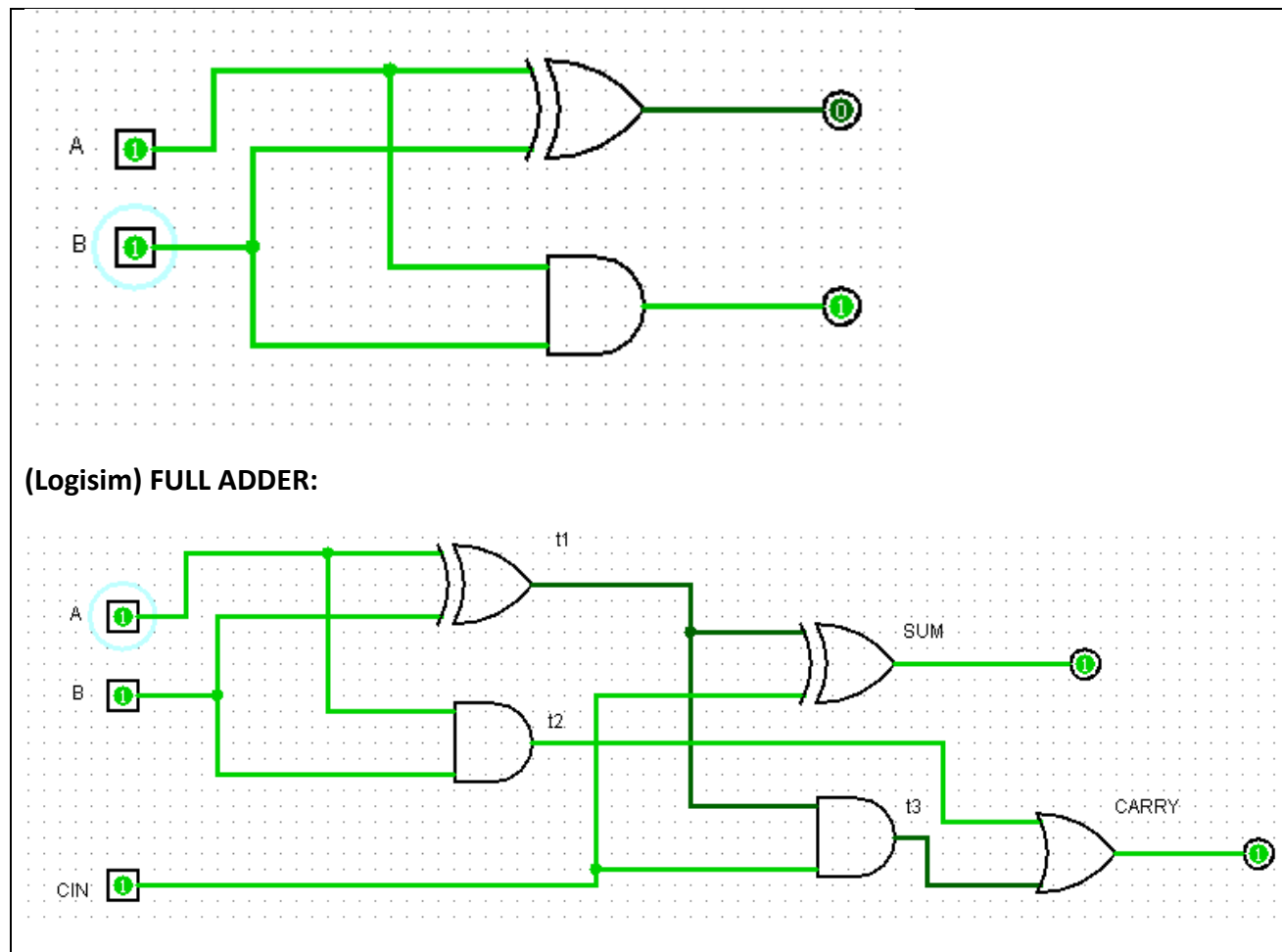
FULL ADDER (Gate Level):



FULL ADDER (Data Flow):



(Logisim) HALF ADDER:



Task 2:

Description:

In this task, we were asked to build 3X8 Decoder, so for that we take the three inputs and apply all the eight combinations to the inputs and then assign one of the eight outputs to them and in Gate Level, we use AND and NOT gate and in Data Flow we assign different input combinations to outputs and the circuit is implemented similarly in Logisim by AND and NOT gate.

Code:

DECODER (Gate Level):

```
module decoder(a,b,c,y0,y1,y2,y3,y4,y5,y6,y7);
input a,b,c;
output y0,y1,y2,y3,y4,y5,y6,y7;
```

```
wire na,nb,nc;

not n1(na,a);

not n2(nb,b);

not n3(nc,c);

and(y0,na,nb,nc); //000

and(y1,na,nb,c); //001

and(y2,na,b,nc); //010

and(y3,na,b,c); //011

and(y4,a,nb,nc); //100

and(y5,a,nb,c); //101

and(y6,a,b,nc); //110

and(y7,a,b,c); //111

endmodule;
```

DECODER (Data Flow):

```
module decoder(a,b,c,y0,y1,y2,y3,y4,y5,y6,y7);

input a,b,c;

output y0,y1,y2,y3,y4,y5,y6,y7;

wire na,nb,nc;

not n1(na,a);

not n2(nb,b);

not n3(nc,c);

assign y0= ~a&~b&~c; //000

assign y1= ~a&~b&c; //001
```

```
assign y2= ~a& b&~c; //010
```

```
assign y3= ~a& b& c; //011
```

```
assign y4= a&~b&~c; //100
```

```
assign y5= a&~b& c; //101
```

```
assign y6= a& b&~c; //110
```

```
assign y7= a& b& c; //111
```

```
endmodule;
```

Common testbench (DECODER):

```
module tb_decoder();
```

```
reg x,y,z;
```

```
wire d0,d1,d2,d3,d4,d5,d6,d7;
```

```
decoder uut(x,y,z,d0,d1,d2,d3,d4,d5,d6,d7);
```

```
initial
```

```
begin
```

```
x=0; y=0; z=0;
```

```
#50;
```

```
x=0; y=0; z=1;
```

```
#50;
```

```
x=0; y=1; z=0;
```

```
#50;
```

```
x=0; y=1; z=1;
```

```
#50;
```

```
x=1; y=0; z=0;
```

```
#50;
```

```
x=1; y=0; z=1;
```

```
#50;
```

```
x=1; y=1; z=0;
```

```
#50;
```

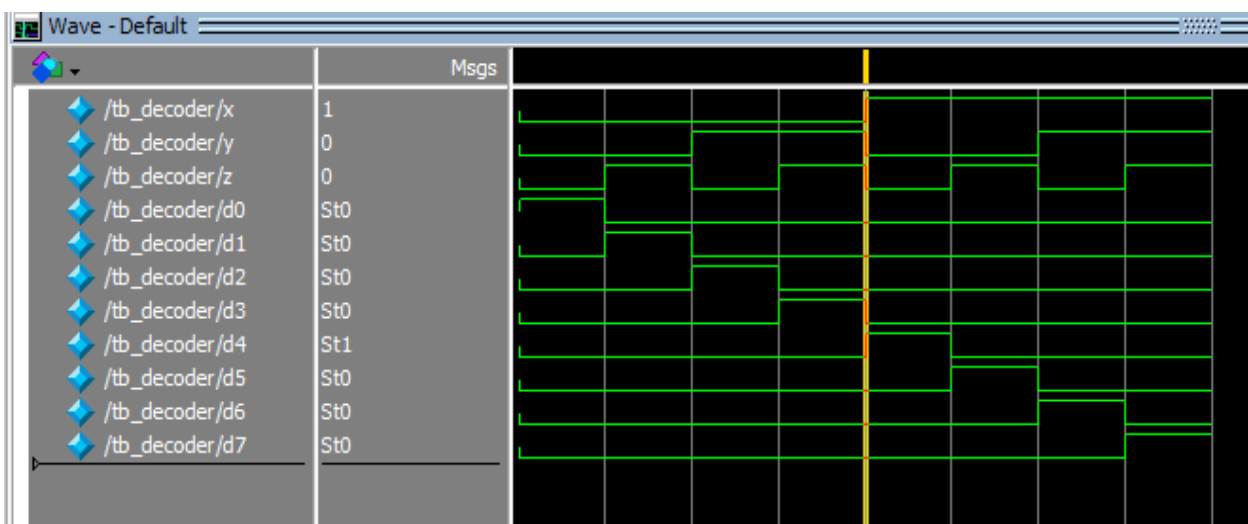
```
x=1; y=1; z=1;
```

```
end
```

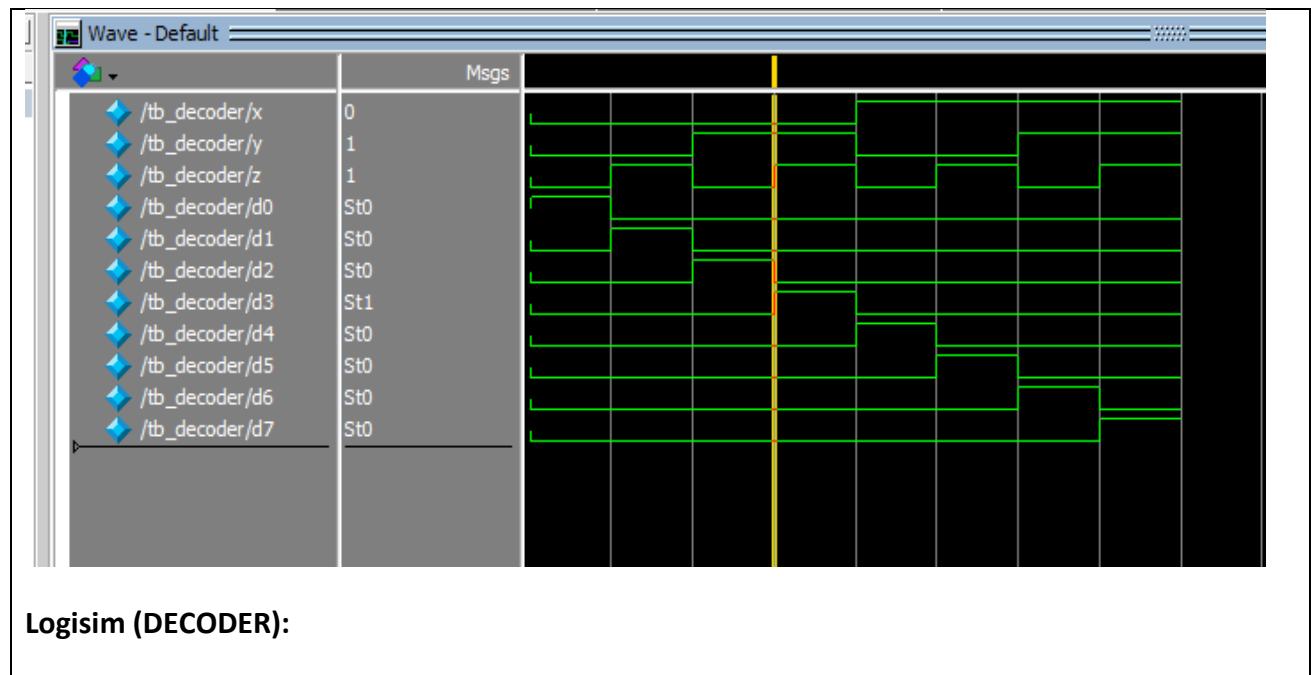
```
endmodule;
```

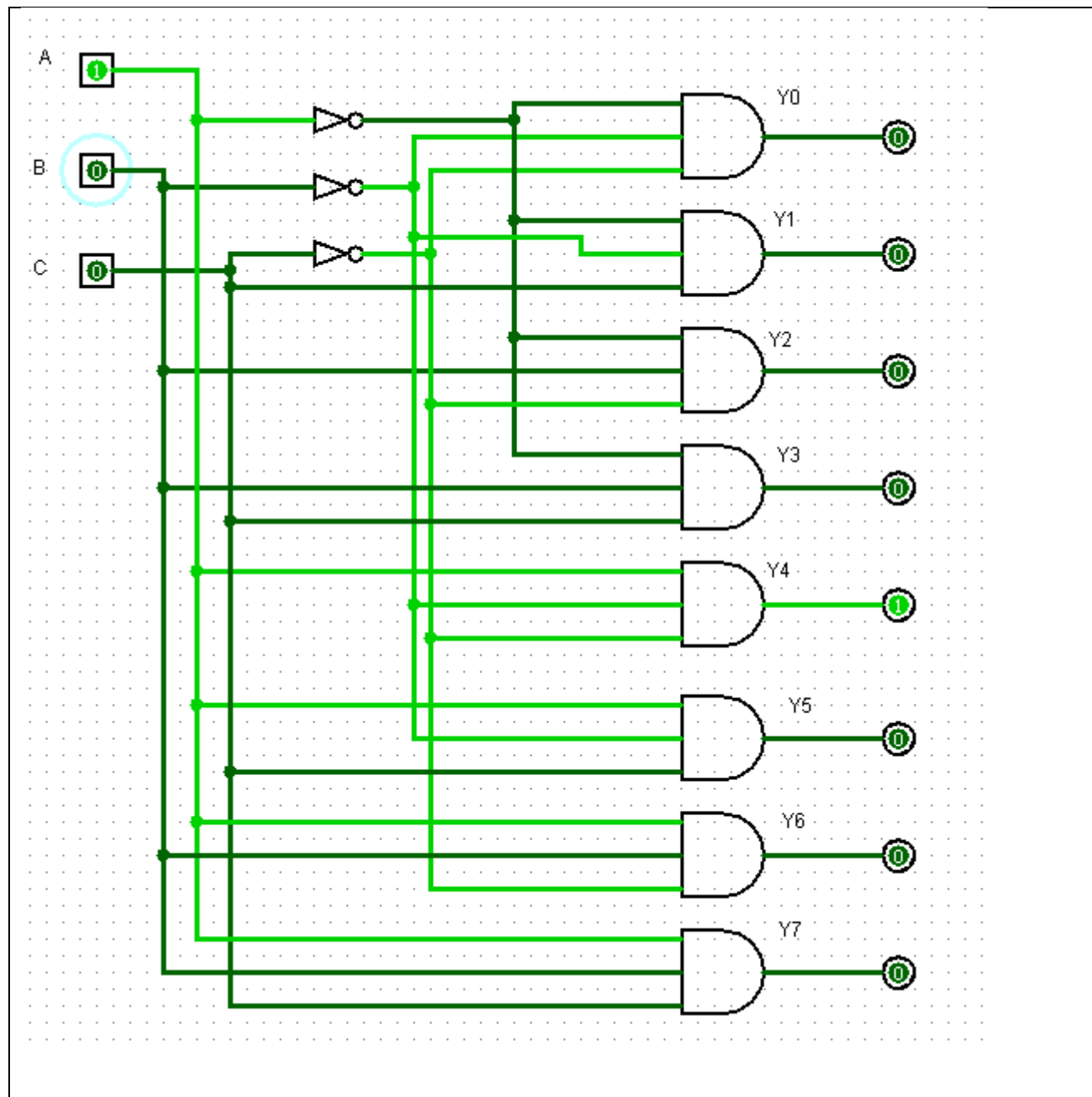
Output:

DECODER (Gate Level):



DECODER (Data Flow):





Task 3:

Description:

In this task, we were asked to build 8X1 MUX, so for that we use three selection lines and in Gate Level, we basically AND all the possible combinations of selection lines and all the eight inputs and OR them to one single output. Meanwhile, in Data Flow, we use the ternary operator by AND all the combinations for selection lines for respective inputs and assign y as output to them and Logisim circuit is implemented by this method.

Code:

MUX (Gate Level):

```
module MUX(a0,a1,a2,a3,a4,a5,a6,a7,s0,s1,s2,y);  
  
input a0,a1,a2,a3,a4,a5,a6,a7,s0,s1,s2;  
  
output y;  
  
wire ns0,ns1,ns2;  
  
wire y0,y1,y2,y3,y4,y5,y6,y7;  
  
not n1(ns0,s0);  
not n2(ns1,s1);  
not n3(ns2,s2);  
  
and x1(y0,a0,ns2,ns1,ns0); //000  
and x2(y1,a1,ns2,ns1,s0); //001  
and x3(y2,a2,ns2,s1,ns0); //010  
and x4(y3,a3,ns2,s1,s0); //011  
and x5(y4,a4,s2,ns1,ns0); //100  
and x6(y5,a5,s2,ns1,s0); //101  
and x7(y6,a6,s2,s1,ns0); //110  
and x8(y7,a7,s2,s1,s0); //111  
  
or o1(y,y0,y1,y2,y3,y4,y5,y6,y7);  
  
endmodule;
```

MUX (Data Flow):

```
module MUX(a0,a1,a2,a3,a4,a5,a6,a7,s0,s1,s2,y);  
  
input a0,a1,a2,a3,a4,a5,a6,a7,s0,s1,s2;
```

```
output y;

assign y= (s2==0 & s1==0 & s0==0)? a0:

(s2==0 & s1==0 & s0==1)? a1:

(s2==0 & s1==1 & s0==0)? a2:

(s2==0 & s1==1 & s0==1)? a3:

(s2==1 & s1==0 & s0==0)? a4:

(s2==1 & s1==0 & s0==1)? a5:

(s2==1 & s1==1 & s0==0)? a6: a7;

endmodule;
```

Common testbench (MUX):

```
module tb_MUX();

reg d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2;

wire m;

MUX uut(d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2,m);

initial

begin

d0=1; d1=1; d2=1; d3=1; d4=1; d5=1; d6=1; d7=1;

s0=0; s1=0; s2=0;

#50

s0=0; s1=0; s2=1;

#50

s0=0; s1=1; s2=0;

#50
```

```
s0=0; s1=1; s2=1;

#50

s0=1; s1=0; s2=0;

#50

s0=1; s1=0; s2=1;

#50

s0=1; s1=1; s2=0;

#50

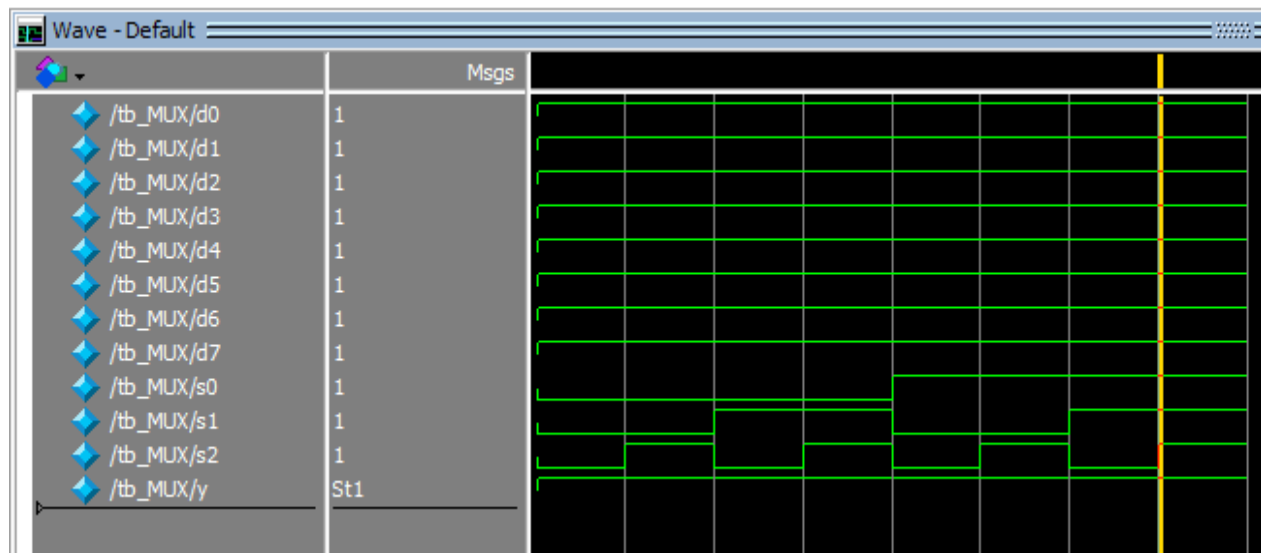
s0=1; s1=1; s2=1;

end

endmodule;
```

Output:

MUX (Gate Level):



MUX (Data Flow):

